Comparing Forecasters for Serverless Invocation Provisioning

Mojtaba Jafari
Department of Computer Engineering
Amirkabir University of Technology
Tehran, Iran
mojtaba.jafari@aut.ac.ir

Ehsan Moradi
Department of Computer Engineering
Amirkabir University of Technology
Tehran, Iran
your.email@aut.ac.ir

Amir Kalbasi
Department of Computer Engineering
Amirkabir University of Technology
Tehran, Iran
kalbasi@aut.ac.ir

***Abstract*—Serverless platforms keep warm containers ready for a changing arrival rate. Too few containers cause cold starts, and too many sit idle. This paper compares six forecasters of per-minute function invocations on a public Azure Functions trace: three simple baselines, seasonal Holt–Winters, a pretrained time-series model, and a small LSTM. Functions are grouped into sparse, daily, bursty, and other workloads. Each forecast covers the next 60 minutes, and the fitted models see the previous day. Error is reported beside a provisioning score that counts short and extra container-minutes, assuming one warm container serves 10 invocations a minute. On a typical function, the seasonal naive forecast has the lowest error. Forecasts that sit above the load leave almost no container short and hold many idle containers. A rule that picks the lowest-shortfall model in each class selects Chronos for sparse and daily functions. On later days that rule has a median shortfall of zero and about 1,700 extra container-minutes, while a high percentile of recent load has a median shortfall of half a container-minute and about 120 extra container-minutes. The same order holds when a container serves 1, 5, or 20 invocations a minute.**

***Index Terms*—serverless computing, workload forecasting, cold start, provisioning**

I. Introduction

Serverless computing, epitomized by Function-as-a-Service (FaaS), has transformed modern cloud architectures by offering automatic elasticity, rapid scalability, and fine-grained pay-per-use billing. Under this paradigm, developers deploy stateless event-driven functions while cloud providers assume full responsibility for provisioning and managing the underlying execution infrastructure. However, delivering this seamless abstraction introduces a critical operational dilemma. When incoming request rates surge or previously inactive functions are triggered, initiating a fresh container environment incurs significant cold-start latency—often orders of magnitude longer than function execution itself—jeopardizing Service Level Objectives (SLOs). Conversely, retaining an excessive fleet of pre-warmed containers to buffer against potential demand leads to substantial memory waste and inflated operational expenditures.

To reconcile this latency-versus-cost trade-off, cloud platforms increasingly deploy predictive resource provisioning, anticipating future invocation volume to warm containers proactively. Yet, a fundamental disconnect exists between standard time-series evaluation and practical cloud resource management. Traditional statistical metrics, such as symmetric mean absolute percentage error (sMAPE) or mean squared error (MSE), treat forecast deviations symmetrically. In production environments, however, forecasting errors carry profoundly asymmetric consequences: an under-prediction directly translates into container shortfalls and unmitigated cold starts, whereas an over-prediction merely results in transiently idle warm containers. Consequently, a predictor that achieves the lowest numerical error may yield an inferior operational operating point if its residual errors systematically trigger cold starts.

Compounding this challenge is the extreme heterogeneity inherent in serverless workloads. Large-scale empirical traces, such as those from Microsoft Azure Functions [1], demonstrate that serverless invocations rarely conform to a single distribution. Instead, workloads encompass heavily skewed behaviors: millions of ultra-sparse functions invoked only a few times per day, structured functions with rigid diurnal seasonality, and highly erratic, bursty event processors. Evaluating forecasting models via fleet-wide aggregate errors obscures these vital distinctions, hiding which algorithmic approach genuinely excels for a specific workload profile.

In this work, we conduct a systematic, capacity-aware empirical comparison of invocation forecasters across representative serverless workload archetypes. Rather than proposing a new autoscaling controller, we investigate the fundamental relationship between time-series predictor design, forecast error, and actual container provisioning performance. We evaluate six diverse forecasting strategies—ranging from simple, training-free heuristics (last-value, trailing percentiles, and seasonal naive) to classical time-series smoothing (seasonal Holt–Winters), deep recurrent architectures (LSTM), and modern zero-shot pretrained foundation models (Chronos-Bolt [2])—across 80 diverse functions sampled from the public Azure Functions trace.

Specifically, this paper makes the following contributions:

- **Workload-Aware Comparative Evaluation:** We categorize serverless functions into four empirical archetypes (sparse, daily-periodic, bursty, and general) and benchmark six forecasting models under a consistent 60-minute prediction horizon with 24 hours of historical context.

xxxx-..-....-....-x/xx/$xx.00 ©2026IEEE

- **Capacity-Centric Provisioning Metrics:** Beyond conventional sMAPE, we introduce a capacity-aware provisioning metric that translates invocation forecasts directly into short and extra container-minutes, exposing the critical divergence between statistical accuracy and physical provisioning efficiency.
- **Practical Selection Heuristics and Insights:** We analyze an empirical model-selection heuristic calibrated on validation days and evaluated on held-out test data. Our findings demonstrate that while pretrained foundation models like Chronos can eliminate container shortfalls, they incur massive over-provisioning overhead; conversely, simple seasonal naive heuristics often achieve superior statistical fidelity and well-balanced operational costs.

The remainder of this paper is organized as follows: Section II reviews related work in serverless autoscaling and workload prediction. Section III details the workload characterization, model architectures, and provisioning metrics. Section IV presents the experimental results, capacity sensitivity analysis, and study limitations. Finally, Section V concludes the paper.

II. Related Work

A. Workload Characterization and Keep-Alive Strategies

A prerequisite for effective serverless resource allocation is understanding real-world invocation patterns. In their seminal characterization of Microsoft Azure Functions, Shahrad et al. demonstrated that serverless workloads exhibit extreme skewness: the vast majority of functions are invoked infrequently, inter-arrival times span several orders of magnitude, and memory consumption remains concentrated in a small subset of applications [1]. To curb cold starts while limiting idle resource consumption, they proposed a practical hybrid keep-alive policy that tracks idle-time histograms and falls back to a time-series predictor when inter-invocation intervals exceed observed thresholds. Building upon these observations, recent efforts such as Lambda-Warmer [3] employ measurement-driven adaptation to tune keep-alive intervals dynamically based on empirical container reuse patterns rather than static timeouts. While keep-alive heuristics effectively capture warm container reuse during active phases, they remain fundamentally reactive when confronted with prolonged silence or abrupt traffic shifts, underscoring the necessity of proactive invocation forecasting.

B. Predictive Autoscaling and Proactive Schedulers

To anticipate arrival surges before cold starts occur, recent research integrates predictive modeling directly into the serverless autoscaling loop. Rather than assessing forecasting models in isolation, modern frameworks couple predictions with custom scaling controllers. For instance, Zhang et al. introduced AAPA [4], an archetype-aware predictive autoscaler on Kubernetes that decomposes invocation histories into spike, periodic, ramp, and stationary patterns, mapping each archetype to a tailored autoscaling policy with uncertainty quantification. Daraghmeh et al. framed container scaling as a classification problem, leveraging machine learning classifiers to determine whether to load or unload containers based on historical feature engineering [5].

Beyond pure classification, hybrid and control-theoretic methodologies have gained traction. Shiekhani et al. combined ARIMA with gradient-boosted decision trees to pre-warm containers and schedule functions on OpenWhisk [6], while Nguyen et al. integrated Fourier series forecasting with model predictive control (MPC) to guarantee proactive container readiness under latency constraints [7]. Similarly, deep neural architectures have been deployed to predict cold-start events directly, subsequently modulating provisioned concurrency levels [8]. While these studies illustrate the potential of prediction-guided autoscaling, they typically evaluate holistic controllers where the raw performance of the forecasting engine is entangled with platform-specific scheduling heuristics, bin packing, or container reuse policies. In contrast, our study isolates the forecaster layer under a standardized, workload-classified protocol, systematically interrogating how alternative forecasters translate invocation predictions into physical container capacity gaps.

C. Time-Series Forecasters: From Heuristics to Foundation Models

The landscape of time-series forecasting in systems research spans a spectrum from lightweight empirical heuristics to sophisticated neural architectures. For periodic and predictable workloads, domain-specific baselines such as seasonal naive repetition and classical exponential smoothing (e.g., Holt–Winters ETS) remain widely used due to their minimal computational overhead and interpretability. For complex non-linear patterns, deep recurrent models such as Long Short-Term Memory (LSTM) networks have long represented the standard learned approach, though they require extensive historical data and specialized training pipelines.

Most recently, the emergence of pretrained time-series foundation models has challenged the convention of training task-specific predictors. Ansari et al. introduced Chronos [2], a probabilistic framework that tokenizes real-valued time series and trains language-model architectures across massive, heterogeneous corpora to achieve zero-shot forecasting on unseen domains. Its high-throughput variant, Chronos-Bolt, emits multi-horizon quantile forecasts in a single forward pass, while Chronos-2 extends these capabilities to multivariate settings with exogenous covariates [9]. In serverless provisioning, upper quantiles offer an attractive mathematical mechanism to hedge against cold-start shortfalls by deliberately pre-warming surplus capacity. However, whether such zero-shot foundation models genuinely surpass lightweight, domain-tuned heuristics—and at what resource cost in idle container-minutes—remains an open question that this empirical investigation resolves.

III. Setup

The trace records invocations per minute for 14 days [1]. Class statistics use days 1–10 only. A missing day is treated

as 1,440 zeros. A function is sparse when at least 90% of those minutes are zero. Otherwise it is daily when the autocorrelation at a lag of one day exceeds 0.30. A remaining function is bursty when its coefficient of variation is at least 2.05, the 75th percentile among non-sparse functions. The rest are labeled other. We draw 20 eligible functions from each class with seed 42. A function is eligible when it is invoked at least once in days 1–10 and at least once in days 11–14. The trace contains 68,394 functions in the first ten days, of which 54,502 are sparse.

Each forecast predicts the next 60 minutes. A new forecast starts every 60 minutes across days 11–14, so the 96 windows cover those four days without overlap. Days 1–10 are history. Days 11–12 are used only to choose the class rule. Days 13–14 are the held-out check. Seasonal Holt–Winters, Chronos, and the LSTM each see the previous 1,440 minutes, one full day. Last-value and the trailing percentile use only the latest minutes, so a longer history would not change them.

Six point forecasts are compared. Seasonal naive repeats the same hour of the previous day when the training day-lag autocorrelation exceeds 0.30, and otherwise repeats the previous 60 minutes. Last-value repeats the latest observed minute. The trailing percentile holds the 95th percentile of the previous 10 minutes for the whole hour. ETS is additive Holt–Winters with a daily season. Its smoothing weights are chosen on days 1–10 only, separately for each function, on a grid of level, trend, and seasonal coefficients, by one-step squared error. The chosen weights then filter the series, and a forecast uses only the state from minutes before its origin. Chronos is the median forecast of `amazon/chronos-bolt-small`, applied with no training on this trace. The LSTM is one shared network with a single layer of 32 units. It is trained on windows from days 1–8, early-stopped on days 9–10, and kept at the weights with validation MAE 8.94. Each LSTM window is scaled by the mean of its own context and scaled back after prediction.

The provisioning score assumes that one warm container serves 10 invocations in a minute. For a forecast $\hat{y}_t$ of the actual count y_t,

$$S = \sum_t \max(0,\, \lceil y_t/10 \rceil - \lceil \hat{y}_t/10 \rceil)\,, \tag{1}$$

$$O = \sum_t \max(0,\, \lceil \hat{y}_t/10 \rceil - \lceil y_t/10 \rceil)\,. \tag{2}$$

S counts short container-minutes. O counts extra container-minutes. Forecasts are clipped at zero before either sum, so a negative prediction prepares no containers. A forecast error that stays inside the same container adds nothing to either sum: eight invocations against a forecast of five still needs one container. Any forecast in $(0, 10]$ prepares one container, so a small positive value on an idle minute costs a full container-minute. Forecast error is the symmetric mean absolute percentage error. For one function it is the mean over test minutes of

$$\mathrm{sMAPE}_t = 100 \cdot \frac{2|y_t - \hat{y}_t|}{|y_t| + |\hat{y}_t|}, \tag{3}$$

and a minute where both values are zero contributes nothing. Tables report the median of that per-function mean. A few heavy functions dominate class sums, which is why the median is the summary.

IV. Evaluation

Table I summarizes the four test days. Seasonal naive has the lowest overall median sMAPE, 7.1%. The gap is largest on daily functions, where its median sMAPE is 3.9%, against 40.0% for the LSTM, 60.6% for seasonal Holt–Winters, and 125.0% for Chronos. Those three models all observed the previous day. Fig. 1 shows the pattern the copy uses. On the daily example, yesterday's shape follows the load. The trailing percentile sits in steps above it. On the sparse example, arrivals are isolated spikes. The LSTM has the lowest sparse sMAPE, 0.7%, and no idle containers, because it predicts zeros. On daily functions its median container shortfall is 1,088, against 45 for seasonal naive.

Container shortfall reorders the same forecasts. Chronos has the smallest overall median, 1.0 short container-minute, because the forecast sits above the load, and it pays 3,281 extra container-minutes. The trailing percentile is next among the point forecasts that an operator might actually run, at 18.5 short container-minutes and 729 extra. Seasonal naive is short 75 container-minutes and wastes 80.5. The LSTM, which predicts low, is short 814. On daily functions both Chronos and the trailing percentile have a median shortfall of zero; Chronos wastes 3,549 container-minutes there and the percentile wastes 4,386.5. Raising Chronos from the median to the 90th percentile brings the overall median shortfall from 1.0 to 0 and raises extra container-minutes from 3,281 to 4,770. A median of zero is not coverage of every function. Chronos is short on 52.5% of functions, the trailing percentile on 66.2%, seasonal naive on 87.5%, and the LSTM on all 80. The 90th percentile is still short on 30.0%. Fig. 2 shows the median trade-off. The upper quantiles lie on the zero-shortfall line.

The selection rule uses only point forecasts. On days 11–12, each class takes the forecast with the lowest median shortfall, then the lower waste, then the lower sMAPE. Sparse functions select Chronos: its median shortfall is 0, against 8.5 for seasonal naive. Daily functions select Chronos as well. The trailing percentile also has a daily shortfall of 0 on those days, and Chronos wastes less, 1,672 container-minutes against 2,113.5. Bursty and other functions select the trailing percentile. Table II applies that frozen rule to days 13–14. Across all functions the rule's median shortfall is 0 and its median waste is 1,713.5 container-minutes. Always using Chronos matches the zero shortfall and wastes 1,920. Always using the trailing percentile has a median shortfall of 0.5 and wastes 120.5. Always using seasonal naive has a median shortfall of 12 and wastes 15. On the held-out sparse functions, seasonal naive also has a shortfall of 0, with 0.5 extra container-minutes, while the rule's Chronos choice wastes 2,703.5. This rule is a shortfall-first heuristic, not a cost-minimizing policy. It spends idle containers to buy a gap that the later days do not repeat.

TABLE I
MEDIAN SMAPE, SHORT CONTAINER-MINUTES, AND EXTRA CONTAINER-MINUTES ON DAYS 11–14.

Model	Metric	Sparse	Daily	Bursty	Other	All
Seasonal naive	sMAPE (%)	0.9	3.9	41.0	12.9	7.1
	Short	13.0	45.0	718.0	150.0	75.0
	Extra	13.0	43.0	719.0	150.5	80.5
Last value	sMAPE (%)	0.9	37.8	48.2	11.9	19.2
	Short	17.0	1150.0	763.5	99.5	410.0
	Extra	0.0	31.0	593.5	93.5	63.5
Trailing percentile	sMAPE (%)	3.8	134.4	74.6	12.9	24.7
	Short	15.0	0.0	143.0	0.0	18.5
	Extra	87.5	4386.5	2183.0	98.0	729.0
ETS	sMAPE (%)	112.8	60.6	116.1	15.7	66.4
	Short	3.5	21.0	250.0	0.0	15.0
	Extra	3225.5	1981.5	2939.0	1046.0	2101.5
LSTM	sMAPE (%)	0.7	40.0	43.7	41.7	40.0
	Short	19.0	1088.0	787.5	1152.0	814.0
	Extra	0.0	13.0	459.5	122.5	46.0
Chronos	sMAPE (%)	178.6	125.0	114.2	12.0	119.4
	Short	0.5	0.0	276.0	0.0	1.0
	Extra	5126.0	3549.0	2760.5	694.0	3281.0

TABLE II
HELD-OUT MEDIANS ON DAYS 13–14. THE RULE USES CHRONOS FOR SPARSE AND DAILY FUNCTIONS AND THE TRAILING PERCENTILE OTHERWISE. THE OTHER ROWS USE ONE FORECAST FOR EVERY CLASS.

Policy	Class	sMAPE (%)	Short	Extra
Selection rule	Sparse	187.7	0.0	2703.5
	Daily	149.7	0.0	1933.0
	Bursty	52.5	33.0	722.5
	Other	10.8	0.0	31.5
	All	113.6	0.0	1713.5
Seasonal naive	Sparse	0.1	0.0	0.5
	Daily	4.0	17.5	21.0
	Bursty	14.2	260.5	258.0
	Other	9.3	42.0	39.0
	All	2.7	12.0	15.0
Trailing percentile	Sparse	0.1	2.0	0.0
	Daily	134.5	0.0	2278.5
	Bursty	52.5	33.0	722.5
	Other	10.8	0.0	31.5
	All	16.2	0.5	120.5
Chronos	Sparse	187.7	0.0	2703.5
	Daily	149.7	0.0	1933.0
	Bursty	129.3	78.5	1665.5
	Other	13.5	0.0	332.5
	All	131.2	0.0	1920.0

Table III repeats S and O at capacities of 1, 5, 10, and 20 invocations per container-minute. The order is the same at each value. Chronos has the lowest median shortfall and the highest waste. The trailing percentile stays between Chronos and seasonal naive. The LSTM has the largest shortfall. At a capacity of 1 the shortfalls grow, because a single invocation can require its own container: Chronos is short 35.0 container-minutes and seasonal naive is short 206.0. From 5 through 20 the medians barely move.

The study has five limits. The trace is from 2019 and contains 80 sampled functions, not a live deployment. Eligibility required at least one invocation on days 11–14, so a function that goes silent in the test window never enters the sample. The counts are per minute, not arrival times, so S and O are capacity gaps rather than measured cold starts on a platform such as OpenWhisk. Medians hide the heavy functions, which is why the share of functions with any shortfall is reported beside them. The selection rule uses only two days.

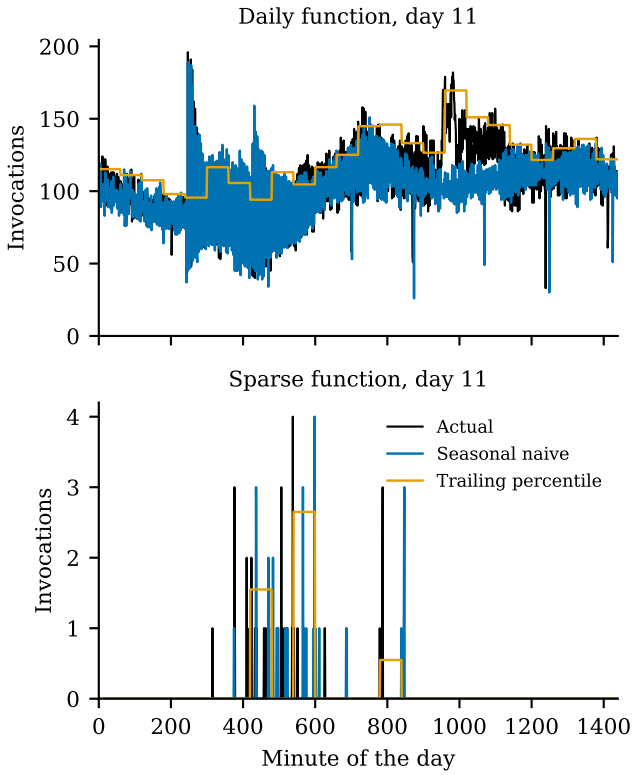


Fig. 1. Day 11 for one daily function and one sparse function. Seasonal naive follows the daily shape. The trailing percentile stays above that shape and reacts at the sparse spikes.

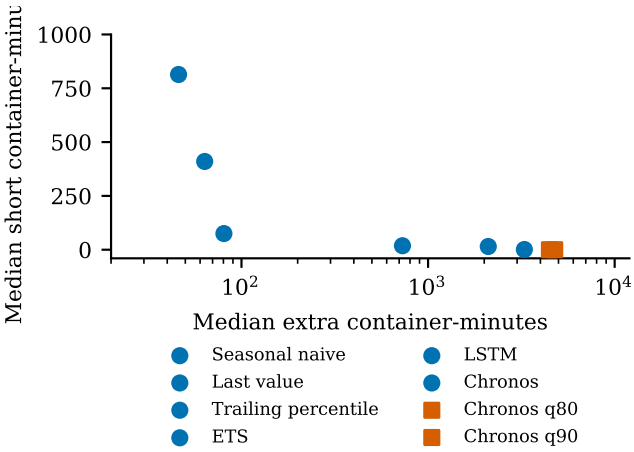


Fig. 2. Median short container-minutes against median extra container-minutes on days 11–14. The horizontal axis is logarithmic. Orange squares are Chronos forecasts at the 80th and 90th percentiles. Closer to the lower left is better.

V. CONCLUSION

On this trace, the seasonal naive forecast is the accurate one. It repeats the previous day when the training day-lag correlation exceeds 0.30, and otherwise repeats the previous hour. That rule is especially accurate for functions whose

TABLE III
MEDIAN SHORT AND EXTRA CONTAINER-MINUTES ON DAYS 11–14 WHEN ONE CONTAINER SERVES C INVOCATIONS A MINUTE.

Model	**Metric**	**1**	**5**	**10**	**20**
Seasonal naive	Short	206.0	75.0	75.0	75.0
	Extra	233.0	80.5	80.5	80.5
Last value	Short	916.0	432.0	410.0	389.5
	Extra	205.0	63.5	63.5	63.5
Trailing percentile	Short	52.0	19.0	18.5	16.5
	Extra	3205.5	919.5	729.0	699.0
ETS	Short	46.5	15.0	15.0	15.0
	Extra	3692.0	2582.5	2101.5	1959.5
LSTM	Short	1151.5	927.5	814.0	785.0
	Extra	595.5	46.0	46.0	46.0
Chronos	Short	35.0	1.5	1.0	1.0
	Extra	4367.5	3398.5	3281.0	3237.0

shape repeats every day. Seasonal Holt–Winters, Chronos-Bolt, and a small LSTM all receive that same day and remain well above the copy on error. Measured in containers, the high forecasts are the ones with almost no shortfall, and they pay for it with idle containers. A shortfall-only rule selects Chronos for sparse and daily functions. On held-out days its median shortfall is zero and its median waste is about 1,700 container-minutes, while the trailing percentile is short by half a container-minute and wastes about 120. A next measurement is to replace the capacity assumption with cold starts recorded on a real platform.

REFERENCES

[1] M. Shahrad, R. Fonseca, I. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, Jul. 2020, pp. 205–218.

[2] A. F. Ansari, L. Stella, C. Turkmen, X. Zhang, P. Mercado *et al.*, "Chronos: Learning the language of time series," *Transactions on Machine Learning Research*, 2024. [Online]. Available: https://openreview.net/forum?id=gerNCVqqtR

[3] Y. Singh and R. Shrestha, "Lambda-warmer: A measurement-driven adaptive framework for cost-aware cold-start mitigation in AWS lambda," *Future Generation Computer Systems*, vol. 185, p. 108681, 2026.

[4] G. Zhang, S. Vippagunta, R. Nandagopal, S. Raman, J. Xu, M. Pfeiffer *et al.*, "AAPA: An archetype-aware predictive autoscaler with uncertainty quantification for serverless workloads on Kubernetes," *arXiv preprint arXiv:2507.05653*, 2025.

[5] M. Daraghmeh, Y. Jararweh, and A. Agarwal, "Leveraging machine learning and feature engineering for optimal data-driven scaling decision in serverless computing," *Simulation Modelling Practice and Theory*, vol. 140, p. 103090, 2025.

[6] L. Shiekhani, H. Wang, W. Shi, J. Liu, Y. Qiu, C. Gu, and W. Ding, "The hybrid model: Prediction-based scheduling and efficient resource management in a serverless environment," *Applied Sciences*, vol. 15, no. 14, p. 7632, 2025.

[7] C. Nguyen, M. Bhuyan, and E. Elmroth, "Taming cold starts: Proactive serverless scheduling with model predictive control," *arXiv preprint arXiv:2508.07640*, 2025.

[8] N. S. Kumar and S. S. Samy, "Cold start prediction and provisioning optimization in serverless computing using deep learning," *Concurrency and Computation: Practice and Experience*, 2025.

[9] A. F. Ansari, O. Shchur, J. Küken, A. Auer, B. Han, P. Mercado *et al.*, "Chronos-2: From univariate to universal forecasting," *arXiv preprint arXiv:2510.15821*, 2025.